BeiDou Short Message Communication — Gap Analysis

Research Gaps | Problem Statement | Proposed Solution | Method Used & Why | What It Proves / Refutes

#	Research Gap	Why It Is a Problem	Proposed Solution	Method Used & Why Chosen	What It Proves / Refutes
1	**No standardised coordinate encoding scheme for BDS-3 SMC payloads**	ASCII encoding uses ~152–208 bits for data needing only 64 bits — a 69% waste. For GSMC (global, 560-bit limit), a full telemetry struct in ASCII exceeds the single-message limit, forcing fragmentation with no native reassembly guarantee in the BDS protocol.	Implement three modes on ESP32: MODE 0 (ASCII baseline), MODE 1 (Binary 64-bit fixed-point), MODE 3 (Dynamic Huffman ~48 bits). Python decoders verify each format independently.	**Method:** Comparative encoding experiment. Transmit the same coordinate in MODE 0 (ASCII), MODE 1 (Binary), and MODE 3 (Huffman). Record bit count per mode from Serial Monitor. **Why chosen:** Direct hardware measurement is more rigorous than theoretical estimation. Allows real compression ratio on live BDS-3 hardware.	**Proves:** Binary fixed-point encoding reduces BDS-3 coordinate payload by ≥60% vs ASCII — making full telemetry feasible inside GSMC's 560-bit global limit in a single message. **Refutes:** The assumption that ASCII is sufficient for BDS-3 SMC when global GSMC coverage is required.
2	**End-to-end latency of the BDS-SMC → software → UAV chain is uncharacterised**	No study has measured the full chain: ESP32 → BDS module → GEO satellite → GCC → web platform → UAV GCS. Without this budget it is impossible to assess if BDS-3 SMC is fast enough to guide a UAV to a moving survivor (drift >0.5 m/s).	Instrument the full chain: [T1] ESP32 firmware marker at send; [T2] PC arrival via serial_logger.py; [T3] web platform receipt. Run 30 transmissions and compute mean, std dev, min, max latency.	**Method:** Repeated timestamped logging (n=30, 10 s intervals). [T1] marked by ESP32 firmware; [T2] recorded by serial_logger.py. Statistics: mean, std dev, min, max for TX latency and total latency. **Why chosen:** 30 samples satisfies CLT for mean estimation. Timestamps at both ends eliminate clock ambiguity.	**Proves:** First empirical end-to-end latency characterisation of the BDS-3 SMC chain. Determines whether the ~1–2 s round-trip supports UAV waypoint updates to a moving survivor. **Refutes:** The assumption that BDS-3 total latency is known from individual link studies alone.
3	**No empirical RDSS transmission success-rate study across rescue environments**	RSMC requires clear line-of-sight to GEO satellites (elevation ~38–52° from Hangzhou). In forests, urban canyons, or indoors — common accident environments — signal acquisition is intermittent. The probability of a successful BDS-3 SMC cycle in obstructed terrain is empirically unknown.	Structured field test: 4 environments (open sky, light canopy, urban canyon, indoor) x 20 transmissions each = 80 total. Log success, fail, and latency per attempt. Compute success rate with 95% CI.	**Method:** Controlled repeated-measures field experiment. field_test_logger.py auto-detects [TX#] markers and logs success/fail/latency per attempt. Success rate computed with 95% confidence interval. **Why chosen:** 20 samples per environment gives meaningful CIs. 4 environments span real rescue scenarios. Repetition isolates environment as the independent variable.	**Proves:** Whether environment significantly affects BDS-3 RSMC success rate and by how much. Provides the first empirical BDS-3 SMC acquisition dataset in terrain-obstructed rescue scenarios. **Refutes:** The gap from Springer (2023): that RDSS uplink success rate in obstructed terrain is unknown and unstudied.

#	Research Gap	Why It Is a Problem	Proposed Solution	Method Used & Why Chosen	What It Proves / Refutes
4	**No comparative framework for telemetry encoding schemes across BDS-3 SMC payloads**	UAV rescue telemetry requires multiple fields (lat, lon, alt, battery, mode, flags, timestamp) in one message. No published work compares ASCII, binary struct, Huffman, and AES variants for this payload — leaving protocol designers with no empirical encoding selection guidance.	Implement all 4 formats for a 7-field struct in telemetry_compare.py. Measure bytes, bits, compression ratio vs ASCII, and GSMC single-message compatibility for each. Output gap6_telemetry.csv.	**Method:** Multi-format empirical comparison. Same 7-field telemetry struct (lat, lon, alt, battery, mode, flags, timestamp) encoded in ASCII, Binary, Huffman, and AES-128. Bit counts measured and GSMC fit assessed against the 560-bit hard limit in Python. **Why chosen:** Running all formats on identical data eliminates confounds. Python implementation mirrors ESP32 firmware — results are directly comparable to hardware measurements.	**Proves:** Which encoding scheme best balances data density, GSMC compatibility, and complexity for full UAV rescue telemetry. Provides evidence-based encoding selection guidance — the exact contribution the 2022–2024 BDS compression literature recommended. **Refutes:** The implicit assumption that ASCII is adequate for multi-field BDS-3 SMC telemetry under global GSMC constraints.